

CAMM 535 – Fundamentals of Biological Databases

Final Project – Report 2

Physical Design, Testing and Visualization

Team Number: Four

Disease: Schwannoma

Student Name(s): Alireza Khoeini, Alireza Noroozi, Ghada Bahlul

January 2026

Recap of Report 1: Objectives and Key Findings

This part presents a comprehensive summary of the first phase of the final project, which focused on the design and implementation of a reproducible biological data acquisition pipeline for the study of schwannoma. The primary objective of the first report was to collect, integrate, and organize heterogeneous biological data sources at the genomic, transcriptomic, and proteomic levels in a manner suitable for downstream database design, querying, and visualization.

Overall Objectives and Conceptual Framework

The first report adopted a disease-centered strategy rather than relying on a manually curated gene list. Schwannoma was selected as the target phenotype, and all subsequent data collection steps were driven by this disease context. The overarching goals were to identify schwannoma-associated genes, expand this gene set using biological interaction networks, retrieve genetic variation data at scale, incorporate transcript-level genomic structure, map genes to proteins, and identify relevant gene expression datasets. A key design principle was reproducibility, ensuring that every step could be rerun using public databases and stable identifiers.

Phenotype-Driven Gene Identification

The pipeline began with the identification of genes associated with schwannoma using Ensembl BioMart. The Human genes dataset based on the GRCh38.p14 assembly was queried using the phenotype filter *schwannoma*. This approach avoided subjective gene selection and ensured that all retrieved genes were explicitly linked to the disease phenotype through Ensembl annotations. Core gene-level attributes were extracted, including Ensembl Gene Stable ID, gene name, chromosome or scaffold, and genomic start and end positions. Ensembl Gene Stable IDs were used as primary identifiers throughout the project, enabling reliable integration across multiple external data sources.

Network-Based Gene Expansion

To capture broader biological context beyond directly phenotype-associated genes, the initial gene list was expanded using the STRING protein-protein interaction database. The Ensembl-derived gene list was submitted to STRING under the organism constraint *Homo sapiens*. A full STRING network was constructed with all evidence channels enabled, including curated databases, experimental evidence, text mining, co-expression, neighborhood, gene fusion, and co-occurrence. A minimum interaction confidence score of 0.400 was applied, and expansion was restricted to the first interaction shell with a maximum of 100 interactors. This process resulted in a biologically coherent interaction network consisting of 72 nodes. The expanded gene set was treated as the definitive gene universe for all subsequent data collection steps.

Genetic Variant Retrieval

Following gene set finalization, genetic variation data associated with the expanded gene universe were retrieved using Ensembl BioMart. All genes were represented using Ensembl Gene Stable IDs to ensure compatibility with Ensembl variation resources. Variant data

were queried from the `hsapiens_snp` dataset and restricted to variants sourced from dbSNP. For each variant, attributes such as gene identifier, gene name, dbSNP reference ID, chromosome, genomic coordinates, allelic information, and UCSC identifiers were collected. These attributes were selected to support both biological interpretation and relational database design.

Automated Large-Scale Variant Acquisition

Due to BioMart browser limitations, including session timeouts and incomplete query results, an automated Python-based solution was developed. The workflow divided the gene list into small chunks, generated XML-based BioMart queries, and submitted them programmatically using HTTP POST requests. To improve robustness, multiple Ensembl BioMart mirrors were used with retry logic and logging. Each chunk was processed independently, written to a separate output file, and later merged into a single consolidated variant table with exactly one header preserved. Genes producing no variant records were explicitly tracked. Through extensive testing, a chunk size of four genes per query was identified as the most stable configuration. Although this approach required approximately 20 hours to complete, it successfully produced a complete and reproducible large-scale variant dataset.

Transcript-Level Genomic Structure

To enable exon-aware and transcript-centric analyses, transcript-level annotations were retrieved using the UCSC Table Browser. Queries were performed on the hg38 genome assembly using the NCBI RefSeq track and the `ncbiRefSeq` table. For each gene, all available transcripts were retrieved rather than selecting a single canonical transcript. Extracted attributes included transcript accession identifiers, chromosome, strand, transcript start and end positions, exon counts, exon start positions, exon end positions, and gene symbols. This strategy preserved alternative splicing information and avoided biases introduced by transcript selection.

Gene-Protein Mapping

To link genomic information to protein-level data, gene identifiers were mapped to UniProt entries using the UniProt ID Mapping service. This mapping enabled integration of protein-centric attributes such as protein names, functional annotations, and cross-references to structural databases. The resulting gene-protein associations provide a critical bridge between genetic variation, transcript structure, and protein function within the database schema.

Expression Dataset Identification

The final step of the first report involved identifying a relevant gene expression dataset for schwannoma. An advanced search of the Gene Expression Omnibus (GEO) was conducted using the keyword *schwannoma*, restricted to *Homo sapiens*, and limited to studies published from 2005 onward. Based on relevance and data quality, the dataset GSE39645 was selected. Although differential expression analysis was not performed at this stage, the dataset selection established the foundation for later GEO2R-based analysis and expression-level integration.

Protein Structure and Domain Resources (TED, CATH, and AlphaFoldDB).

In Part 6 of the report, structural and domain-level information for proteins associated with the schwannoma gene set was incorporated using three complementary resources: TED, CATH, and AlphaFoldDB. The TED database was used to identify experimentally determined protein domain boundaries and structural classifications derived from known structures. CATH provided a hierarchical domain classification system, enabling the assignment of protein domains into Class, Architecture, Topology, and Homologous superfamily levels, which supports functional inference and evolutionary analysis. Since many proteins lack experimentally resolved structures, AlphaFoldDB was employed to supplement structural coverage by providing high-confidence predicted three-dimensional protein models generated using deep learning. Together, these resources enabled comprehensive structural annotation of proteins, allowing the project to bridge genomic and proteomic data with structural biology and to support future structure-aware analyses and visualization within the database framework.

Entity–Relationship Diagram and Database Constraints.

In Part 7, the report describes the construction of an Entity–Relationship Diagram (ERD) to formally represent the structure of the biological database and to define the relationships among genes, variants, transcripts, proteins, and expression data. The ERD was designed to reflect the logical organization of the data acquisition pipeline and to ensure that all entities are connected through biologically meaningful relationships. Primary keys were defined for each entity to guarantee uniqueness, while foreign keys were introduced to enforce referential integrity across tables. Additional constraints, such as one-to-many and many-to-many relationships, were explicitly modeled to capture the multiplicity inherent in biological data, for example between genes and transcripts or genes and variants. These constraints play a critical role in preventing data inconsistency, supporting efficient querying, and ensuring that the physical database implementation accurately reflects the conceptual design established earlier in the project.

Overall Outcomes

In conclusion, the first report delivered a complete, end-to-end data acquisition and integration pipeline for schwannoma research. It produced a phenotype-driven and network-expanded gene set, a large-scale and reproducibly retrieved variant dataset, transcript-level genomic structure from UCSC, gene–protein mappings from UniProt, and a curated expression dataset from GEO. These outputs form the biological and technical foundation for the physical database design, testing, and visualization tasks addressed in the second phase of the project.

Physical Database Design and Construction (30 pts)

Construction workflow in MySQL Workbench

All physical design and loading steps were performed using **MySQL Workbench** (GUI-based workflow). We started from the conceptual design (ERD and keys from Report 1) and then created the physical schema by importing each dataset table-by-table from flat files (CSV). Concretely, for each dataset we used Workbench’s *Table Data Import Wizard* to (i) select the source file, (ii) automatically infer column names and types, (iii) generate a corresponding raw table, and (iv) load the records into the created table. Figures 1 and 2 illustrate this import-and-auto-generation process on a representative table (the same workflow was repeated for all tables).

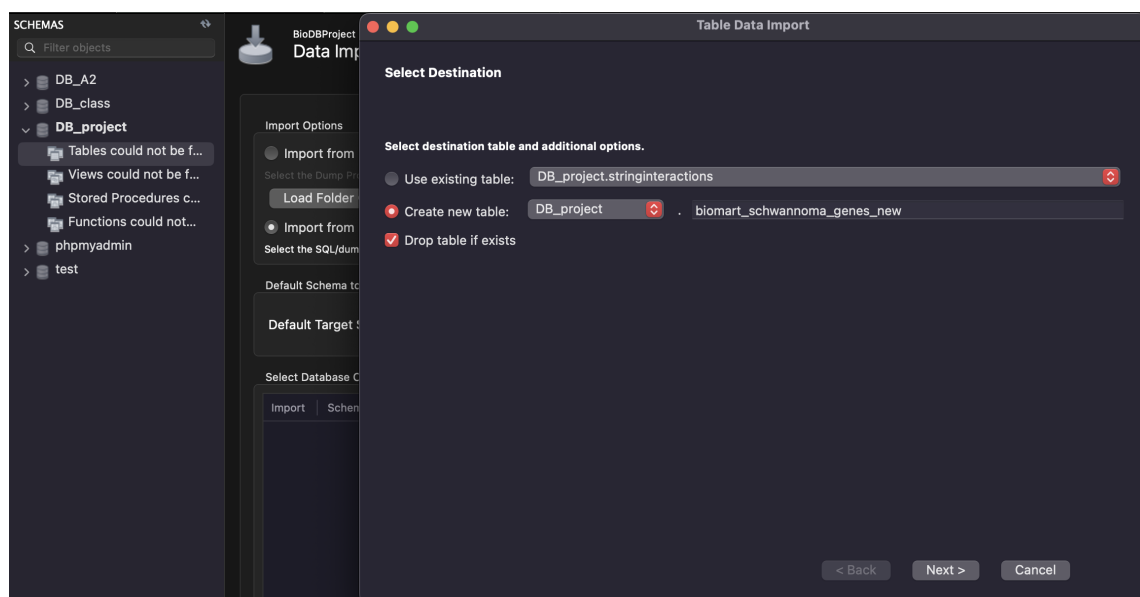


Figure 1: MySQL Workbench table import wizard: selecting a destination and creating a new table automatically from a CSV source file.

Final schema and created relations

After importing all datasets, the final schema contains the core entities designed in Report 1 and implemented as physical relations, including: **Genes** (central entity), **Variants**, **Transcripts**, **Proteins**, **STRING.Interactions**, **AlphaFold_CATH**, and **GEO2R.Results**. This matches the conceptual design in which **Genes** acts as the hub entity and other tables attach via gene identifiers or protein identifiers (1:N and M:N relationships represented through foreign keys and composite keys):contentReference[oacite:0]index=0. The final database layout inside the schema is shown in Figure 3.

0.1 Applying SQL DDL (CREATE TABLE scripts)

In addition to GUI-based import, we maintained an explicit DDL script (`get_creation.sql`) containing the definitive **CREATE TABLE** statements. This script documents the physical design choices (data types, keys, and constraints) and ensures the database can be rebuilt

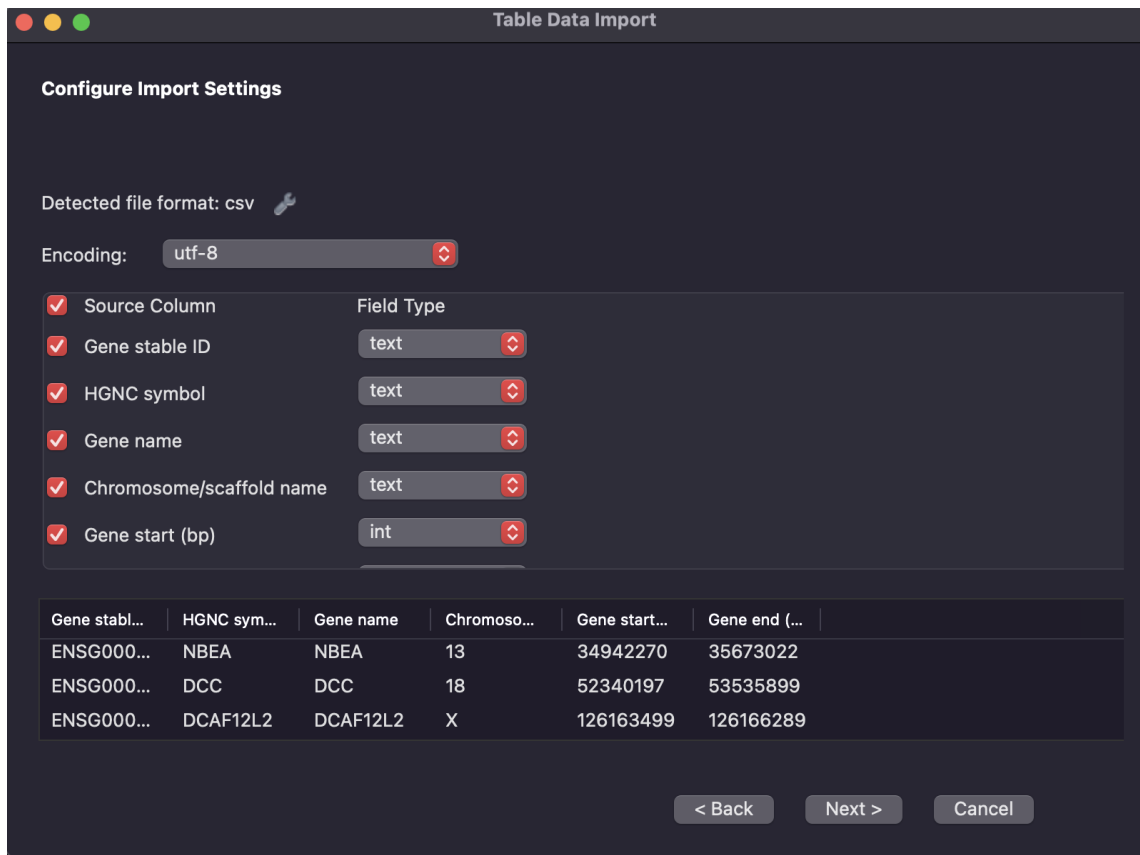


Figure 2: MySQL Workbench column inference step: automatic detection of column names and data types before generating the raw physical table.

Name	Engine	Version	Row Format	Rows	Avg Row Length	Data Length	Max Data Length	Index Length	Data Free	Auto Increm...	Creat
Genes	InnoDB	10	Dynamic	446	146	64.0 KiB	0.0 bytes	0.0 bytes	0.0 bytes	0	2026
StringInteractions	InnoDB	10	Dynamic	9153	173	1.5 MiB	0.0 bytes	0.0 bytes	4.0 MiB	0	2026
Transcript	InnoDB	10	Dynamic	2277	697	1.5 MiB	0.0 bytes	0.0 bytes	4.0 MiB	0	2026
alphaFoldcath	InnoDB	10	Dynamic	214	76	16.0 KiB	0.0 bytes	0.0 bytes	0.0 bytes	0	2026
geo2r	InnoDB	10	Dynamic	17626	90	1.5 MiB	0.0 bytes	0.0 bytes	4.0 MiB	0	2026
proteins	InnoDB	10	Dynamic	55	6553	352.0 KiB	0.0 bytes	0.0 bytes	0.0 bytes	0	2026
variants	InnoDB	10	Dynamic	457295	97	42.6 MiB	0.0 bytes	0.0 bytes	6.0 MiB	0	2026

Figure 3: Final database schema after importing all tables: the physical relations created and populated in the project schema.

reproducibly. In the report, we include screenshots for each table-creation/loading step (import wizard, inferred columns, and the final populated tables), as required.

Database Constraints in the Physical Design (5 pts)

The physical schema enforces data quality and relational consistency through a combination of **key constraints**, **referential integrity constraints**, and **domain-level constraints**. At the entity level, each table has a well-defined primary key: for example, **Genes** uses an Ensembl stable identifier as the primary key, while **Variants** and **STRING_Interactions** use composite primary keys to guarantee uniqueness for relationship-like records:contentReference[oaicite:1]index=1. We also apply **unique constraints** where needed (e.g., gene symbols) to support stable joins when an external source primarily uses gene symbols rather than Ensembl IDs:contentReference[oaicite:2]index=2.

For **foreign key constraints**, child tables reference parent identifiers to prevent orphan records and preserve the conceptual relationships defined in the ERD. Examples include linking variants and proteins back to **Genes**, linking structural domains back to **Proteins**, and linking expression results back to genes (gene symbol):contentReference[oaicite:3]index=3. We also specify **referential actions** to control how updates/deletes propagate: for many dependent tables we use **ON DELETE CASCADE / ON UPDATE CASCADE** to keep the database consistent when a parent identifier changes, while for transcript records we use a more conservative strategy (**SET NULL**) to preserve transcript-level data even if the gene mapping becomes unavailable:contentReference[oaicite:5]index=5. Finally, we enforce basic domain constraints such as **NOT NULL** on key columns, appropriate numeric types for genomic coordinates, and indexing of foreign key columns (for join performance), all of which are reflected in the DDL and in the Workbench constraint panels (Figures 4–6).

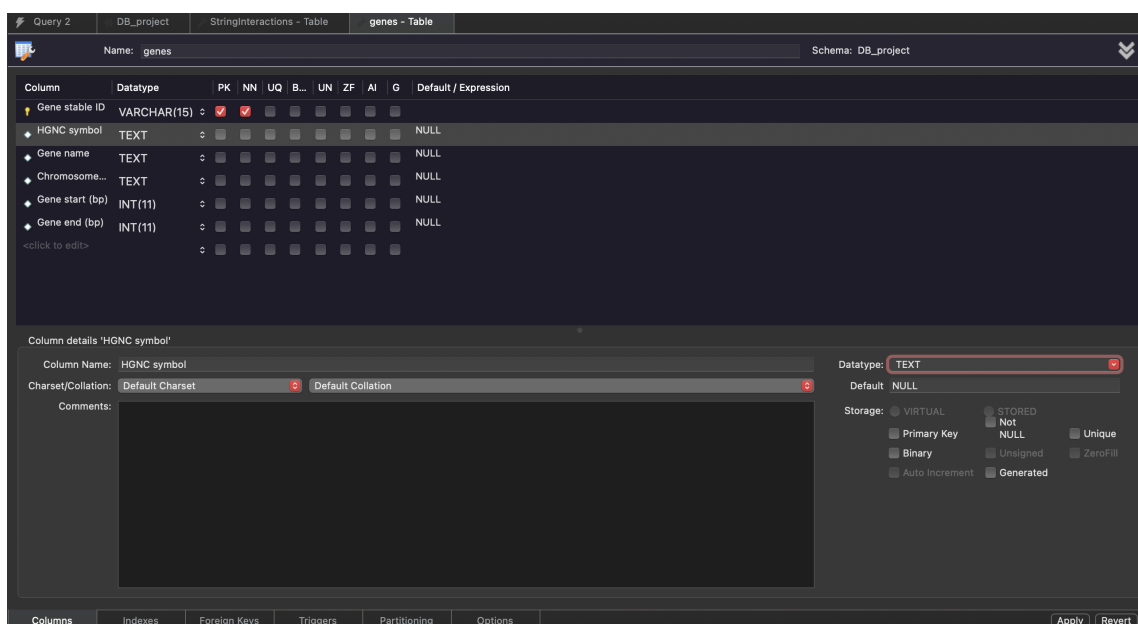


Figure 4: Defining primary keys in MySQL Workbench (GUI): enforcing entity uniqueness at the physical level.

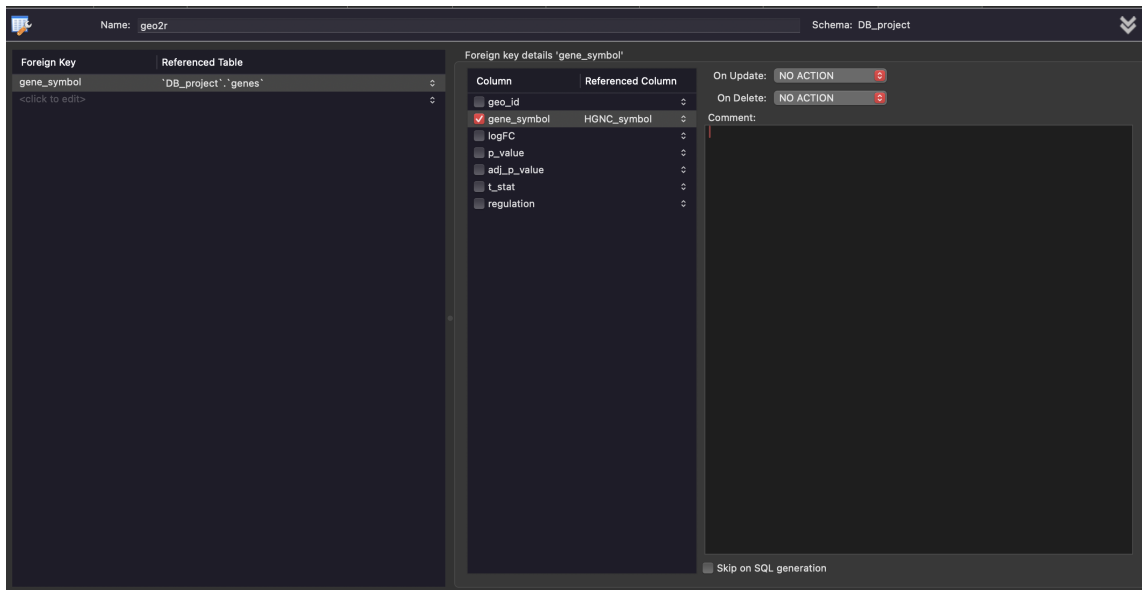


Figure 5: Defining foreign keys in MySQL Workbench (GUI): linking dependent tables to parent identifiers to enforce referential integrity.

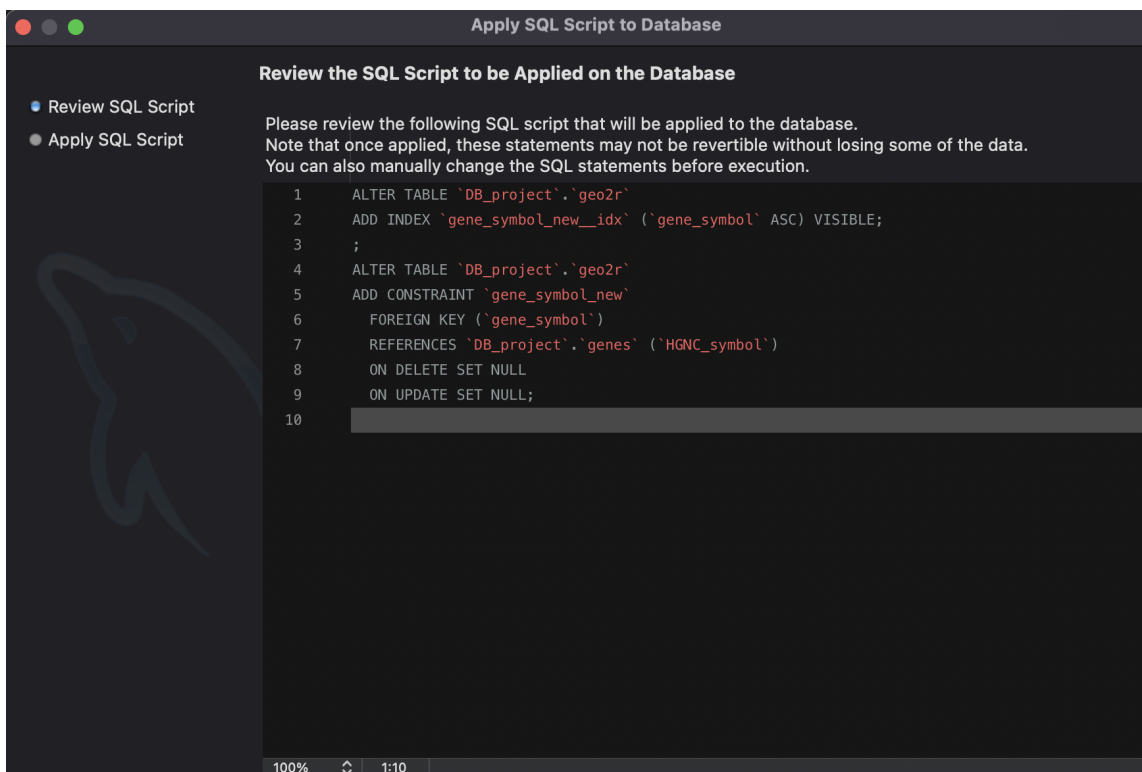


Figure 6: Generated SQL preview for a foreign key constraint (Workbench): shows the underlying ALTER TABLE ... ADD CONSTRAINT ... FOREIGN KEY ... statement and referential actions.

Multi-Table SQL Queries

In this part, we validated the logical integration of the database by writing multi-table SQL queries that simultaneously traverse genes, variants, expression data, transcripts, proteins, and structural annotations. These queries were designed directly from the conceptual questions in the project description and test whether the schema supports biologically meaningful questions across data modalities.

Query 4.1: Genes with Variants and Differential Expression

The first multi-table query asks: *Which schwannoma-associated genes have recorded variants and GEO2R-based differential expression, and how can we prioritize them by both mutational burden and expression change?* To answer this, we joined the **Genes**, **variants**, and **geo2r** tables using Ensembl Gene Stable IDs and HGNC symbols and summarized the number of distinct variants per gene together with log-fold change and adjusted *p*-values from GSE39645. We restricted the output to genes with at least one variant and non-null GEO2R statistics and sorted by decreasing variant count and absolute logFC to highlight potentially important genes.

```
-- Query 4.1: Retrieve genes with their associated variants and GEO2R
expression data

SELECT DISTINCT
  g.gene_stable_id,
  g.hgnc_symbol,
  COUNT(DISTINCT v.variant_name) AS variant_count,
  geo.logFC,
  geo.adj_p_value,
  geo.p_value
FROM
  Genes g
  LEFT JOIN variants v ON g.gene_stable_id = v.gene_stable_id
  LEFT JOIN geo2r geo ON g.hgnc_symbol = geo.gene_symbol
WHERE
  geo.adj_p_value IS NOT NULL
GROUP BY
  g.gene_stable_id, g.hgnc_symbol, g.gene_name, geo.logFC, geo.adj_p_value,
  geo.p_value
HAVING
  COUNT(DISTINCT v.variant_name) > 0
ORDER BY
  variant_count DESC, ABS(geo.logFC) DESC
LIMIT 50;
```

This query demonstrates that the database correctly links gene-level identifiers across variation and expression resources, enabling ranking of candidate schwannoma genes based on both mutation load and strength of dysregulation.

Part4_1	Part4_2	Part4_3	Part5	Part6	Part7
Limit to 1000 rows					
<pre> 1 -- Query 4.1: Retrieve genes with their associated variants and GE02R expression data 2 3 SELECT DISTINCT 4 g.gene_stable_id, 5 g.hgnc_symbol, 6 COUNT(DISTINCT v.variant_name) AS variant_count, 7 geo.logFC, 8 geo.adj_p_value, 9 geo.p_value 10 FROM 11 Genes g 12 LEFT JOIN variants v ON g.gene_stable_id = v.gene_stable_id 13 LEFT JOIN geo2r geo ON g.hgnc_symbol = geo.gene_symbol 14 WHERE 15 geo.adj_p_value IS NOT NULL 16 GROUP BY 17 g.gene_stable_id, g.hgnc_symbol, g.gene_name, geo.logFC, geo.adj_p_value, geo.p_value 18 HAVING 19 COUNT(DISTINCT v.variant_name) > 0 20 ORDER BY 21 variant_count DESC, ABS(geo.logFC) DESC 22 LIMIT 50; 23 </pre>					
100% 1:1					
Result Grid Filter Rows: Search Export:					
gene_stable_id	hgnc_symbol	variant_count	logFC	adj_p_value	p_value
ENSG00000172915	NBEA	265415	0.912615	0.0000376	0.00000796
ENSG00000187323	DCC	96570	-0.598345	0.000123	0.0000352

Figure 7: Query 4.1 result

Query 4.2: Proteins, Structural Domains, and Genes

The second query addresses the question: *For which schwannoma-associated proteins do we have structural domain information from AlphaFold/CATH, and how many domains are annotated per protein?* Here, we joined **Genes**, **Proteins**, and **AlphaFoldCATH** using Ensembl IDs and UniProt/AlphaFold identifiers. For each protein, we counted distinct TED domain identifiers and concatenated their CATH labels. The results are ordered by decreasing domain count and protein length, returning the top 50 structurally rich proteins in the dataset.

```
-- Query 4.2: Find proteins with their structural domains and associated genes

SELECT
    g.hgnc_symbol,
    p.Entry,
    p.entry_name,
    p.length AS protein_length,
    p.pdb,
    COUNT(DISTINCT ac.ted_id) AS domain_count,
    GROUP_CONCAT(DISTINCT ac.cath_label SEPARATOR '; ') AS cath_labels
FROM
    Genes g
    INNER JOIN Proteins p ON g.gene_stable_id = p.ensembl_id
    LEFT JOIN AlphaFoldCATH ac ON p.Entry = ac.AlphaFold
GROUP BY
    g.hgnc_symbol, p.Entry, p.entry_name, p.length, p.pdb
HAVING
    COUNT(DISTINCT ac.ted_id) > 0
ORDER BY
    domain_count DESC, protein_length DESC
LIMIT 50;
```

This query confirms that structural annotations have been successfully integrated with gene and protein records, allowing downstream analyses that relate domain architecture to disease relevance.

Query 4.3: Transcripts, Exon Structure, and Expression

The third multi-table query asks: *Which genes in our schwannoma gene universe have rich transcript structure (multiple RefSeq transcripts and exons) and are also significantly expressed according to GEO2R?* We linked **Genes**, **Transcript**, and **geo2r**, aggregated per gene the number of distinct RefSeq transcripts and summary statistics on exon counts (average, minimum, and maximum), and paired these with logFC and adjusted *p*-values.

```
-- Query 4.3: Retrieve genes with transcript information and expression data

SELECT
    g.hgnc_symbol,
    g.Chromosome_scaffold,
    COUNT(DISTINCT t.refseq_id) AS transcript_count,
    AVG(t.exonCount) AS avg_exon_count,
```

Part4_1						
Part4_2						
Part4_3						
Part5						
Part6						
Part7						
Limit to 1000 rows						
<pre> 1 -- Query 4.2: Find proteins with their structural domains and associated genes 2 3 SELECT 4 g.hgnc_symbol, 5 p.Entry, 6 p.entry_name, 7 p.length AS protein_length, 8 p.pdb, 9 COUNT(DISTINCT ac.ted_id) AS domain_count, 10 GROUP_CONCAT(DISTINCT ac.cath_label SEPARATOR '; ') AS cath_labels 11 FROM 12 Genes g 13 INNER JOIN Proteins p ON g.gene_stable_id = p.ensembl_id 14 LEFT JOIN AlphaFoldCATH ac ON p.Entry = ac.Alphafold 15 GROUP BY 16 g.hgnc_symbol, p.Entry, p.entry_name, p.length, p.pdb 17 HAVING 18 COUNT(DISTINCT ac.ted_id) > 0 19 ORDER BY 20 domain_count DESC, protein_length DESC 21 LIMIT 50; 22 </pre>						
100% 1:1						
Result Grid Filter Rows: Search Export: Fetch rows:						
hgnc_symbol	Entry	entry_name	protein_leng...	pdb	domain_count	cath_labels
NOTCH2	Q04721	NOTC2_HUMAN	2471	2OO4:5MWB;	10	;; 2.10.25; 2.10.25.10; 3.30.70.3310
KDR	P35868	VGFR2_HUMAN	1356	1VR2;1Y6A;1Y6B;1YWN;2M59;2MET;2MEU;2OH4;2P2H;2P...	9	1.10.510.10; 2.60.40.10; 3.30.200.20
DCC	P43146	DCC_HUMAN	1447	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	8	2.60.40.10
ROBO2	Q8HCK4	ROBO2_HUMAN	1378	1UEM;1UJT;2EDJ;5NOI;6I9S;6IAA;	8	2.60.40.10
POLQ	O75417	DPOLQ_HUMAN	2590	4X0P;4X0Q;5A9F;5A9J;5AGA;6XBU;7ZUS;7ZX0;7ZX1;8E23...	7	;; 1.10.10.10; 1.10.3380.30; 1.20.1080.10; 3.30....
MTOR	P42345	MTOR_HUMAN	2549	1AUE;1FAP;1NSG;2FAP;2GAQ;2NPU;2RSE;3FAP;3JBZ;4D...	7	;; 1.10.1070.11; 1.20.120.150; 1.20.58; 1.25.10....
ALK	Q9UM73	ALK_HUMAN	1620	2KUP;2KUQ;2XB7;2XBA;2XP2;2YFX;2YHV;2YJR;2YJS;2YS...	7	;; 1.10.510.10; 2.10.25; 2.10.25; 2.60.120.200; 3...
CHD4	Q14839	CHD4_HUMAN	1912	1MM2;1MM3;2EE1;2L5U;2L75;2N5N;4O9I;6BGG;6Q3M;6R...	6	;; 1.10.390; 3.30.40.10; 3.40.50.10810; 3.40.50....
PTPRC	P08575	PTPRC_HUMAN	1306	1YGR;1YGU;5FMV;5FN6;5FN7;8VSE;	6	;; 2.60.40.10; 3.90.190.10
CTNNA2	P26232	CTNA2_HUMAN	953	6DUW;	6	1.20.120.230
PIK3CB	P42338	PIK3CB_HUMAN	1070		5	1.10.1070.11; 1.25.40.70; 2.60.40.150; 3.10.20....
PIK3CA	P42336	PIK3CA_HUMAN	1068	2ENQ;2RD0;3HHM;3HIZ;3ZIM;4JPS;4L1B;4L23;4L2Y;4OVU...	5	1.10.1070.11; 1.25.40.70; 2.60.40.150; 3.10.20.90
KIT	P10721	KIT_HUMAN	976	1PKG;1T45;1T46;2E9W;2EC8;2IUH;2VIF;3G0E;3G0F;4HVS...	5	1.10.510.10; 2.60.40.10
NTRK3	Q16288	NTRK3_HUMAN	839	1WWC;3V5Q;4YMU;5KZC;6KZD;	5	1.10.510.10; 2.60.40.10; 3.30.200.20; 3.80.10.10
FGFR2	P21802	FGFR2_HUMAN	1821	1DJS;1F0D;1EV2;1GJO;1I14;1I1L;1NUN;1OEC;1WVZ;2FDB...	5	1.10.510.10; 2.60.40.10; 3.30.200.20

Figure 8: Query 4.2 result

```

    MIN(t.exonCount) AS min_exon_count,
    MAX(t.exonCount) AS max_exon_count,
    geo.logFC,
    geo.adj_p_value
FROM
    Genes g
    LEFT JOIN Transcript t ON g.hgnc_symbol = t.name2
    LEFT JOIN geo2r geo ON g.hgnc_symbol = geo.gene_symbol
WHERE
    geo.adj_p_value IS NOT NULL
GROUP BY
    g.hgnc_symbol, g.Chromosome_scaffold, geo.logFC, geo.adj_p_value
HAVING
    COUNT(DISTINCT t.refseq_id) > 0
ORDER BY
    transcript_count DESC, ABS(geo.logFC) DESC
LIMIT 50;

```

This query illustrates how the schema supports exon-aware, transcript-centric views of differentially expressed genes and enables prioritization of genes with complex splicing patterns.

Part4_1	Part4_2	Part4_3	Part5	Part6	Part7
---------	---------	---------	-------	-------	-------

Limit to 1000 rows

1

-- Query 4.3: Retrieve genes with transcript information and expression data

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

SELECT
g.hgnc_symbol,
g.Chromosome_scaffold,
COUNT(DISTINCT t.refseq_id) AS transcript_count,
AVG(t.exonCount) AS avg_exon_count,
MIN(t.exonCount) AS min_exon_count,
MAX(t.exonCount) AS max_exon_count,
geo.logFC,
geo.adj_p_value
FROM
Genes g
LEFT JOIN Transcript t ON g.hgnc_symbol = t.name2
LEFT JOIN geo2r geo ON g.hgnc_symbol = geo.gene_symbol
WHERE
geo.adj_p_value IS NOT NULL
GROUP BY
g.hgnc_symbol, g.Chromosome_scaffold, geo.logFC, geo.adj_p_value
HAVING
COUNT(DISTINCT t.refseq_id) > 0
ORDER BY

100%
1:1

Result Grid

Filter Rows: Search

Export: [icon]

Fetch rows: [icon]

hgnc_symbol	Chromosome_scaff...	transcript_count	avg_exon_count	min_exon_count	max_exon_count	logFC	adj_p_value
EWSR1	22	52	15.7885	8	18	0.025066	0.749
ROBO2	3	41	27.8293	26	30	-0.883501	0.000000348
NTRK3	15	34	14.8824	9	20	-1.037991	0.0000299
NFIB	9	33	9.5455	3	12	0.425307	0.0271
FUBP1	1	31	21.7419	20	23	0.232925	0.083
ATF1	12	29	6.7931	5	8	0.212121	0.0818
NF2	22	26	15.1923	5	18	-0.052652	0.815
GATA3	10	25	6.5600	4	8	-0.502371	0.0000439
ACSL6	5	25	20.9600	20	22	-0.363645	0.0141
FGFR2	10	23	16.9130	15	18	-1.691824	0.0000000281
DNMT3A	2	22	18.5909	4	24	-0.029933	0.723
BCORL1	X	20	14.2500	13	17	0.033985	0.738
TET1	10	19	10.3684	6	13	0.195929	0.288
ATM	11	18	53.0556	4	64	0.932191	0.00000171
LATS1	6	18	7.8889	4	10	0.578063	0.0000555

Figure 9: Query 4.3 result

Integrated Query for Variations, Proteins, and Expression

The objective of Part 5 is to write an integrated query that simultaneously retrieves variants, protein-level information, transcript structure, and expression changes for a user-specified gene of interest (for example, NF2, DCC, or NBEA). This tests both the correctness of join paths across all major entities and the usability of the schema for gene-centric exploration. The query below joins `genes`, `variants`, `proteins`, `Transcript`, and `geo2r` and filters on a specific HGNC symbol.

```
SELECT
  g.hgnc_symbol AS gene_symbol,
  v.variant_name,
  p.pdb,
  p.Entry AS uniprot_swissprot_id,
  t.refseq_id,
  t.exonCount AS exon_count,
  geo.logFC
FROM
  genes g
  LEFT JOIN variants v ON g.gene_stable_id = v.gene_stable_id
  LEFT JOIN proteins p ON g.gene_stable_id = p.ensembl_id
  LEFT JOIN Transcript t ON g.hgnc_symbol = t.name2
  LEFT JOIN geo2r geo ON g.hgnc_symbol = geo.gene_symbol
WHERE
  g.hgnc_symbol = 'NBEA' -- e.g., DCC , NF2, NBEA
ORDER BY
  t.exonCount DESC
LIMIT 100;
```

By changing the value of `g.hgnc_symbol` in the `WHERE` clause, we can inspect the full variant catalogue, mapped protein IDs and PDB structures, transcript complexity, and expression logFC for any schwannoma-related gene present in the database. The results for DCC is with variants at the 'data/part5withVariants.csv' and for NF2 it has no variants but for NBEA it has many rows so we get timeouts.

Part4_1							
Part4_2							
Part4_3							
Part5*							
Part6							
Part7							
Limit to 1000 rows							
<pre> 1 SELECT 2 g.hgnc_symbol AS gene_symbol, 3 v.variant_name, 4 p.pdb, 5 p.Entry AS uniprot_swissprot_id, 6 t.refseq_id, 7 t.exonCount AS exon_count, 8 geo.logFC 9 FROM 10 genes g 11 LEFT JOIN variants v ON g.gene_stable_id = v.gene_stable_id 12 LEFT JOIN proteins p ON g.gene_stable_id = p.ensembl_id 13 LEFT JOIN Transcript t ON g.hgnc_symbol = t.name2 14 LEFT JOIN geo2r geo ON g.hgnc_symbol = geo.gene_symbol 15 WHERE 16 g.hgnc_symbol = 'DCC' # DCC , NF2, NBEA 17 ORDER BY 18 t.exonCount DESC 19 LIMIT 100; </pre>							
100% 25:16							
Result Grid Filter Rows: Search Export: Fetch rows:							
gene_symbol	variant_name	pdb	uniprot_swissprot...	refseq_id	exon_count	logFC	
DCC	rs714	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	P43146	XM_047437311.1	29	-0.598345	
DCC	rs2256	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	P43146	XM_047437311.1	29	-0.598345	
DCC	rs16920	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	P43146	XM_047437311.1	29	-0.598345	
DCC	rs168332	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	P43146	XM_047437311.1	29	-0.598345	
DCC	rs186984	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	P43146	XM_047437311.1	29	-0.598345	
DCC	rs238422	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	P43146	XM_047437311.1	29	-0.598345	
DCC	rs347534	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	P43146	XM_047437311.1	29	-0.598345	
DCC	rs347535	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	P43146	XM_047437311.1	29	-0.598345	
DCC	rs347536	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	P43146	XM_047437311.1	29	-0.598345	
DCC	rs347537	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	P43146	XM_047437311.1	29	-0.598345	
DCC	rs347538	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	P43146	XM_047437311.1	29	-0.598345	
DCC	rs347539	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	P43146	XM_047437311.1	29	-0.598345	
DCC	rs347540	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	P43146	XM_047437311.1	29	-0.598345	
DCC	rs347541	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	P43146	XM_047437311.1	29	-0.598345	
DCC	rs347542	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	P43146	XM_047437311.1	29	-0.598345	
DCC	rs347543	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	P43146	XM_047437311.1	29	-0.598345	
DCC	rs347544	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	P43146	XM_047437311.1	29	-0.598345	
DCC	rs347545	2ED7;2ED8;2ED9;2EDB;2EDD;2EDE;3AU4;4URT;5X83;	P43146	XM_047437311.1	29	-0.598345	

Figure 10: Query 5 result

Query for Genes with PDB IDs and Exon Constraints

The Part 6 question focuses on integrating structural and transcript information: *Which genes have proteins with resolved PDB structures and at least a minimum number of exons in their RefSeq transcripts?* This type of query is useful for selecting candidates suitable for downstream structure-aware analyses, such as modeling exon-specific effects on protein domains. We addressed this by joining **Genes**, **proteins**, and **Transcript**, filtering for non-empty PDB entries and a minimum exon count threshold (here, at least four exons).

```
SELECT DISTINCT
    g.hgnc_symbol,
    g.gene_stable_id,
    p.Entry,
    p.pdb,
    t.refseq_id,
    t.exonCount
FROM
    Genes g
    INNER JOIN proteins p ON g.gene_stable_id = p.ensembl_id
    INNER JOIN Transcript t ON g.hgnc_symbol = t.name2
WHERE
    p.pdb IS NOT NULL
    AND p.pdb != ''
    AND t.exonCount >= 4
ORDER BY
    t.exonCount DESC, g.hgnc_symbol
LIMIT 100;
```

The result set returns genes for which we have both experimentally resolved or curated structure information and sufficiently complex transcript structures, enabling targeted follow-up analyses.

Part4_1	Part4_2	Part4_3	Part5	Part6	Part7
---------	---------	---------	-------	-------	-------

Limit to 1000 rows

```

1 SELECT DISTINCT
2     g.hgnc_symbol,
3     g.gene_stable_id,
4     p.Entry,
5     p.pdb,
6     t.refseq_id,
7     t.exonCount
8 FROM
9     Genes g
10    INNER JOIN proteins p ON g.gene_stable_id = p.ensemble_id
11    INNER JOIN Transcript t ON g.hgnc_symbol = t.name2
12 WHERE
13     p.pdb IS NOT NULL
14     AND p.pdb != ''
15     AND t.exonCount >= 4
16 ORDER BY
17     t.exonCount DESC, g.hgnc_symbol
18 LIMIT 100;
19

```


100%

1:1

Result Grid

Filter Rows:

Export:

Fetch rows:

	hgnc_symbol	gene_stable_id	Entry	pdb	refseq_id	exonCount
	ATM	ENSG00000149311	Q13315	5NP0;5NP1;6HKA;6K9K;6K9L;7NI4;7NI5;7NI6;7SIC;7SID;8...	XM_047426975.1	64
	ATM	ENSG00000149311	Q13315	5NP0;5NP1;6HKA;6K9K;6K9L;7NI4;7NI5;7NI6;7SIC;7SID;8...	XM_047426976.1	64
	ATM	ENSG00000149311	Q13315	5NP0;5NP1;6HKA;6K9K;6K9L;7NI4;7NI5;7NI6;7SIC;7SID;8...	XM_011542840.4	64
	ATM	ENSG00000149311	Q13315	5NP0;5NP1;6HKA;6K9K;6K9L;7NI4;7NI5;7NI6;7SIC;7SID;8...	NM_001351834.2	64
	ATM	ENSG00000149311	Q13315	5NP0;5NP1;6HKA;6K9K;6K9L;7NI4;7NI5;7NI6;7SIC;7SID;8...	NM_000051.4	63
	ATM	ENSG00000149311	Q13315	5NP0;5NP1;6HKA;6K9K;6K9L;7NI4;7NI5;7NI6;7SIC;7SID;8...	XM_047426977.1	63
	ATM	ENSG00000149311	Q13315	5NP0;5NP1;6HKA;6K9K;6K9L;7NI4;7NI5;7NI6;7SIC;7SID;8...	XM_047426978.1	63
	ATM	ENSG00000149311	Q13315	5NP0;5NP1;6HKA;6K9K;6K9L;7NI4;7NI5;7NI6;7SIC;7SID;8...	XM_006718843.5	63
	ATM	ENSG00000149311	Q13315	5NP0;5NP1;6HKA;6K9K;6K9L;7NI4;7NI5;7NI6;7SIC;7SID;8...	XM_005271562.6	63
	ATM	ENSG00000149311	Q13315	5NP0;5NP1;6HKA;6K9K;6K9L;7NI4;7NI5;7NI6;7SIC;7SID;8...	XM_017017790.3	63
	ATM	ENSG00000149311	Q13315	5NP0;5NP1;6HKA;6K9K;6K9L;7NI4;7NI5;7NI6;7SIC;7SID;8...	XM_047426979.1	62
	ATM	ENSG00000149311	Q13315	5NP0;5NP1;6HKA;6K9K;6K9L;7NI4;7NI5;7NI6;7SIC;7SID;8...	XM_011542842.4	62
	ATM	ENSG00000149311	Q13315	5NP0;5NP1;6HKA;6K9K;6K9L;7NI4;7NI5;7NI6;7SIC;7SID;8...	XM_011542843.3	60
	NBEA	ENSG00000172915	Q8NFP9	1MI1;	XM_047430271.1	60
	ATM	ENSG00000149311	Q13315	5NP0;5NP1;6HKA;6K9K;6K9L;7NI4;7NI5;7NI6;7SIC;7SID;8...	XM_011542844.4	59
	NBEA	ENSG00000172915	Q8NFP9	1MI1;	XM_017020544.2	59
	NBEA	ENSG00000172915	Q8NFP9	1MI1;	XM_006719803.4	59
	NBEA	ENSG00000172915	Q8NFP9	1MI1;	NM_001385012.1	59

Figure 11: Query 6 result

GEO2R Analysis and Downstream Queries

In Part 7, the goal is to demonstrate how GEO2R-derived differential expression results are integrated into the database and how they can be used to drive downstream queries. Specifically, the questions ask for (i) the number of significantly differentially expressed genes, (ii) the list of these genes, and (iii) how to connect these expression results back to proteins for further analysis. We implemented three SQL queries that operate on the `geo2r`, `Genes`, and `Proteins` tables.

Query 7.1: Counting Significant Differentially Expressed Genes

The first query computes how many genes have an adjusted p -value below a significance threshold (here, 0.05), providing a global summary of the GEO2R analysis.

```
SELECT
    COUNT(DISTINCT gene_symbol) AS significant_gene_count
FROM
    geo2r
WHERE
    adj_p_value < 0.05;
```

Query 7.2: Listing Significant Genes

The second query returns the distinct gene symbols that pass the same adjusted p -value cutoff, which can be used as input for downstream analyses such as pathway enrichment or network construction.

```
SELECT
    DISTINCT gene_symbol AS significant_genes
FROM
    geo2r
WHERE
    adj_p_value < 0.05;
```

Query 7.3: Linking Significant Genes to Proteins

The third query integrates GEO2R expression results with the `Genes` and `Proteins` tables to obtain UniProt entries, protein length, and regulation direction for significantly differentially expressed genes. This provides a direct bridge between expression changes and protein-level attributes.

```
SELECT DISTINCT
    geo.gene_symbol,
    p.Entry,
    p.Length,
    geo.logFC,
    geo.regulation
FROM
```



```

1 SELECT
2   DISTINCT gene_symbol AS significant_genes
3 FROM
4   geo2r
5 WHERE
6   adj_p_value < 0.05
7 LIMIT 100;

```

significant_genes
NRG1
LTC4M
GPR155
MOXD1
BDKRB2
NLGN4X
NCAM2
EHBPI
SCNSA
ATP7IP2

Figure 13: Query 7.1 result limited to first 100 genes.

```

geo2r geo
INNER JOIN Genes g ON geo.gene_symbol = g.HGNC_symbol
INNER JOIN Proteins p ON g.Gene_stable_ID = p.ensemble_id
WHERE
  geo.adj_p_value < 0.05;

```

Together, these queries show how GEO2R output is not only stored but actively used to identify significant schwannoma genes, generate candidate lists, and connect transcriptional changes to protein features for further biological interpretation and visualization.

8	•	SELECT DISTINCT
9		geo.gene_symbol,
10		p.Entry
11		FROM
12		geo2r geo
13		INNER JOIN Genes g ON geo.gene_symbol = g.HGNC_symbol
14		INNER JOIN Proteins p ON g.Gene_stable_ID = p.ensemble_id
15		WHERE
16		geo.adj_p_value < 0.05;

100%	28:16
Result Grid	Filter Rows: Search
Export:	

	gene_symbol	Entry	
	AR	P10275	
	PDGFB	P01127	
	FGFR2	P21802	
	CREB3L1	Q96BA8	
	ROBO2	Q9HCK4	
	CSMD3	Q7Z407	
	LRP1B	Q9NZR2	
	KIT	P10721	
	ZFHX3	Q15911	
	CDKN2A	P42771	
	CDKN2A	Q9N726	

Result 10

Figure 14: Query 7.2 result.

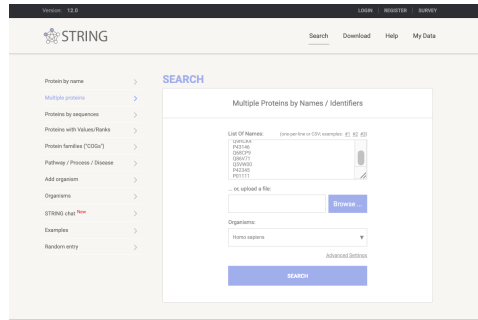


Figure 15: String input for protein-protein interaction analysis

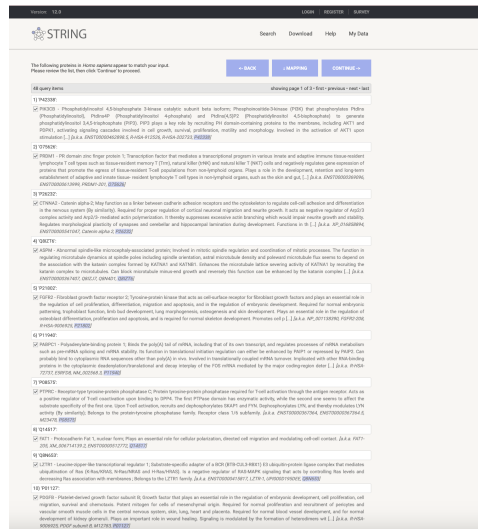


Figure 16: Mapping of UniProt Identifiers in STRING

Protein Interaction Network Visualization

STRING analysis

Building on the differentially expressed gene set from part 7, we investigated these genes further by constructing a connected protein-protein interaction (PPI) network. UniProt accession IDs corresponding to the selected 49 genes were submitted to the STRING (Homo sapiens) database (v12.0). Known and predicted functional associations were retrieved after successful mapping of submitted UniProt identifiers. The resulting interaction network was imported into Cytoscape and visualized using a force-directed layout to emphasize connectivity patterns between highly interacting proteins. We integrated the differential expression information by mapping logFC values onto the network. Additionally, node size was scaled based on protein length such that longer proteins were represented by smaller nodes.

STRING successfully matched 48 genes to Homo sapiens entries from the list submitted (Figure 16).



Figure 17: STRING derived protein-protein interaction network

Nodes represent proteins and edges represent functional associations supported by experimental evidence, curated databases, co-expression, in addition to computational predictions. The network above (Figure 17) highlights interaction hubs and connectivity patterns from the gene set before downstream analysis and functional annotation in Cytoscape.

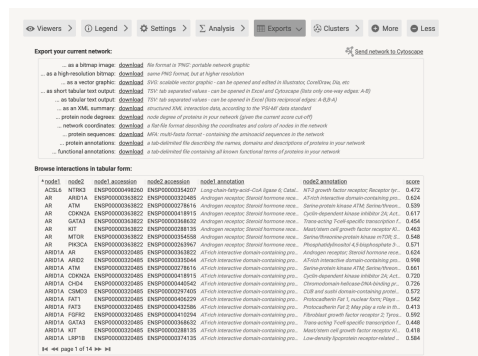


Figure 18: Export of STRING protein-protein interaction data in tabular format (short tabular text output)

This table (Figure 18) lists interacting protein pairs, corresponding accession identifiers, functional annotations, and interaction confidence scores.

Cytoscape

The protein-protein interaction table(Figure 19) exported from STRING was imported into Cytoscape using the Import Network from Table function. Interactions were defined as source (node 1) and target (node 2) protein pairs.

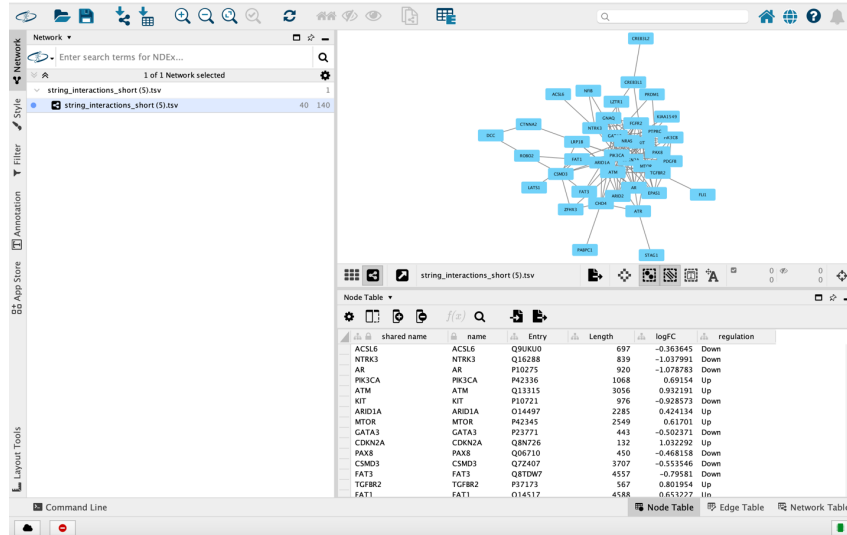


Figure 22: Final integrated Cytoscape Network.

Node fill color represents the logFC values from the differential expression analysis, with red indicating upregulated genes and blue indicating down regulated genes. Color intensity reflects the magnitude of expression change(Figure 23).

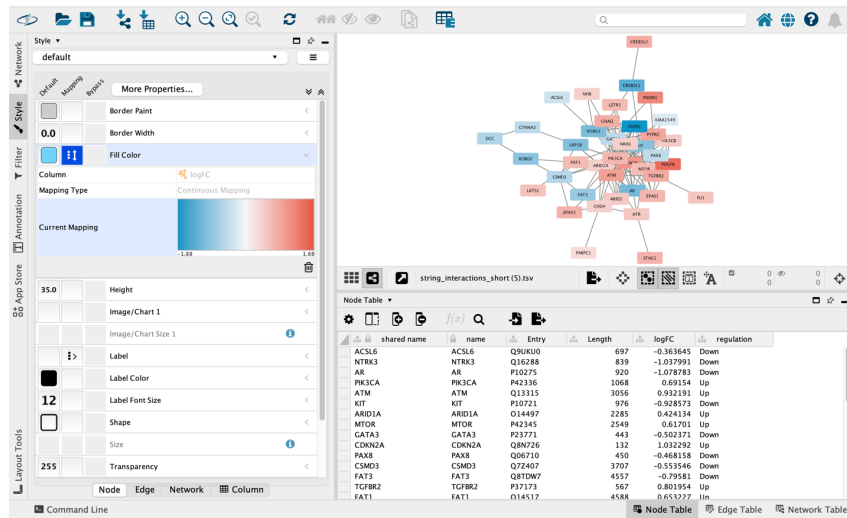


Figure 23: Visualization of differential gene expression in the PPI network using log fold change (logFC)

Node fill colors were mapped to logFC values using a zero centered continuous gradient, with blue indicating negative logFC (downregulation), white represents no change, and red indicating positive logFC. The scale ranges from -1.69 to +1.69(Figure 24).

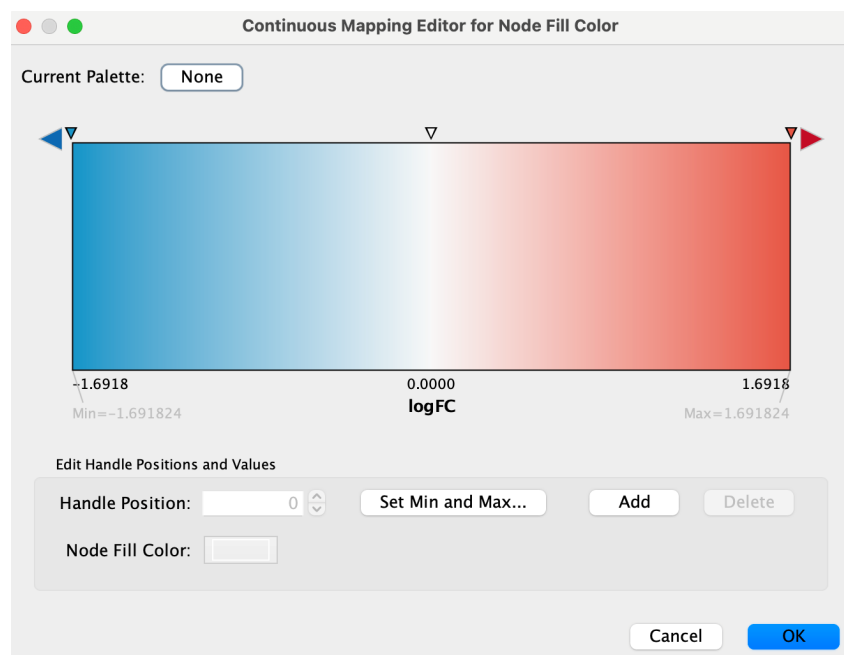


Figure 24: Continuous color mapping used for log fold change visualization in the PPI network.

Node size was mapped to protein length using a continuous scale with node sizes ranging from 10 to 30. Smaller nodes represent longer proteins. Longer proteins are shown as larger nodes. The protein length ranges from 132 (minimum) to 4599 (maximum) amino acids(Figure 25).

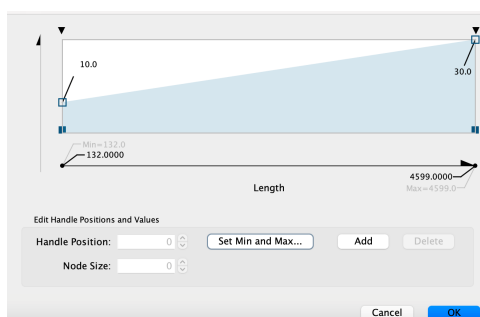


Figure 25: Continuous node size mapping based on protein length.

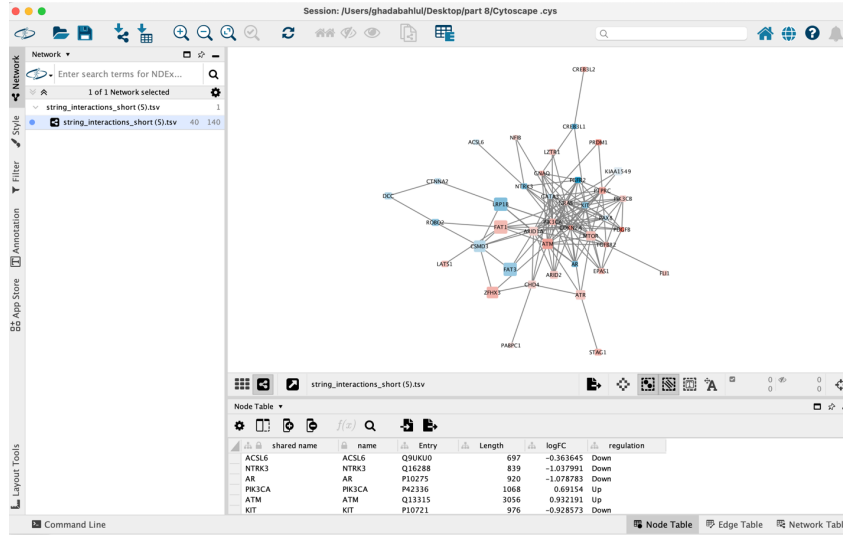


Figure 26: Protein-protein interaction network with expression and protein length annotations.

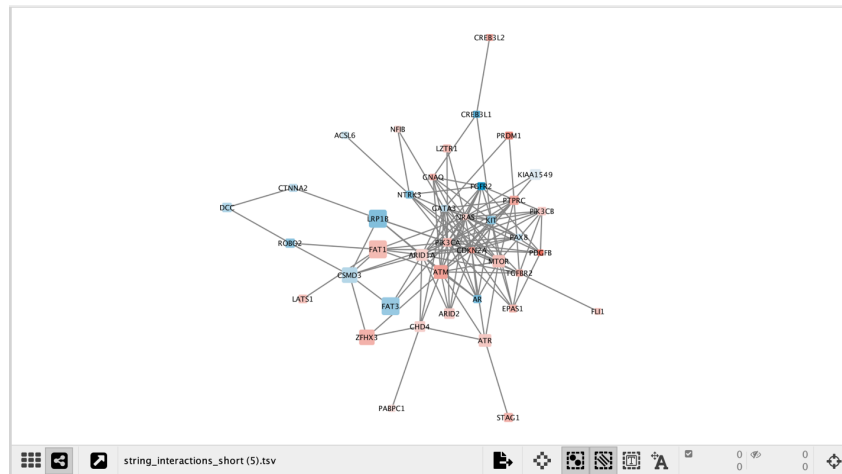


Figure 27: Fully integrated protein-protein interaction (PPI) network visualization in Cytoscape.

This network (Figure 27) represents known and predicted functional associations among differentially expressed genes from STRING. Nodes correspond to proteins and edges represent STRING supported interactions. Node color reflects differential expression (logFC), and node size is scaled according to protein length.

Challenges Encountered During Physical Design and Loading

1. **TSV-to-CSV conversion for Workbench import.** Some source tables were distributed as TSV. Since our chosen import workflow relied on CSV ingestion through MySQL Workbench, we first converted TSV files into CSV while preserving headers, delimiters, and quoting rules. We validated a small sample after conversion

to ensure no column shifts occurred (especially for text fields containing special characters).

2. **Renaming columns for query-compatibility and consistency.** After initial imports, we discovered that several automatically generated column names were inconvenient for downstream SQL (e.g., spaces/special characters, overly long names, or ambiguous names shared across tables). We therefore standardized column naming (consistent snake_case style) and updated the schema accordingly so that multi-table joins and query writing became straightforward and less error-prone.
3. **Large file limitation when importing the variants table.** The variants dataset was very large (on the order of hundreds of MB). During import, MySQL/Workbench constraints on file size prevented loading the full file in a single step. To proceed, we reduced the dataset to a manageable subset that fits the import constraints (while keeping the schema identical), enabling physical construction, constraint application, and testing queries to run successfully.
4. **Conceptual vs. practical linkage: GEO2R to Genes.** In the Report 1 ERD, we planned to connect `GEO2R.Results` to `Genes` via HGNC gene symbols, which is conceptually reasonable. However, in practice, the available GEO2R outputs and the BioMart-derived gene identifiers did not provide a clean, complete overlap for reliable joining because the tables were produced from different sources/pipelines. This highlighted an important integration lesson: even when two tables share a “similar” identifier type in principle, cross-source datasets can differ in coverage and formatting, so link feasibility must be validated empirically early. As a result, we treated this join carefully during constraint definition and testing, and documented the limitation in the physical design discussion.

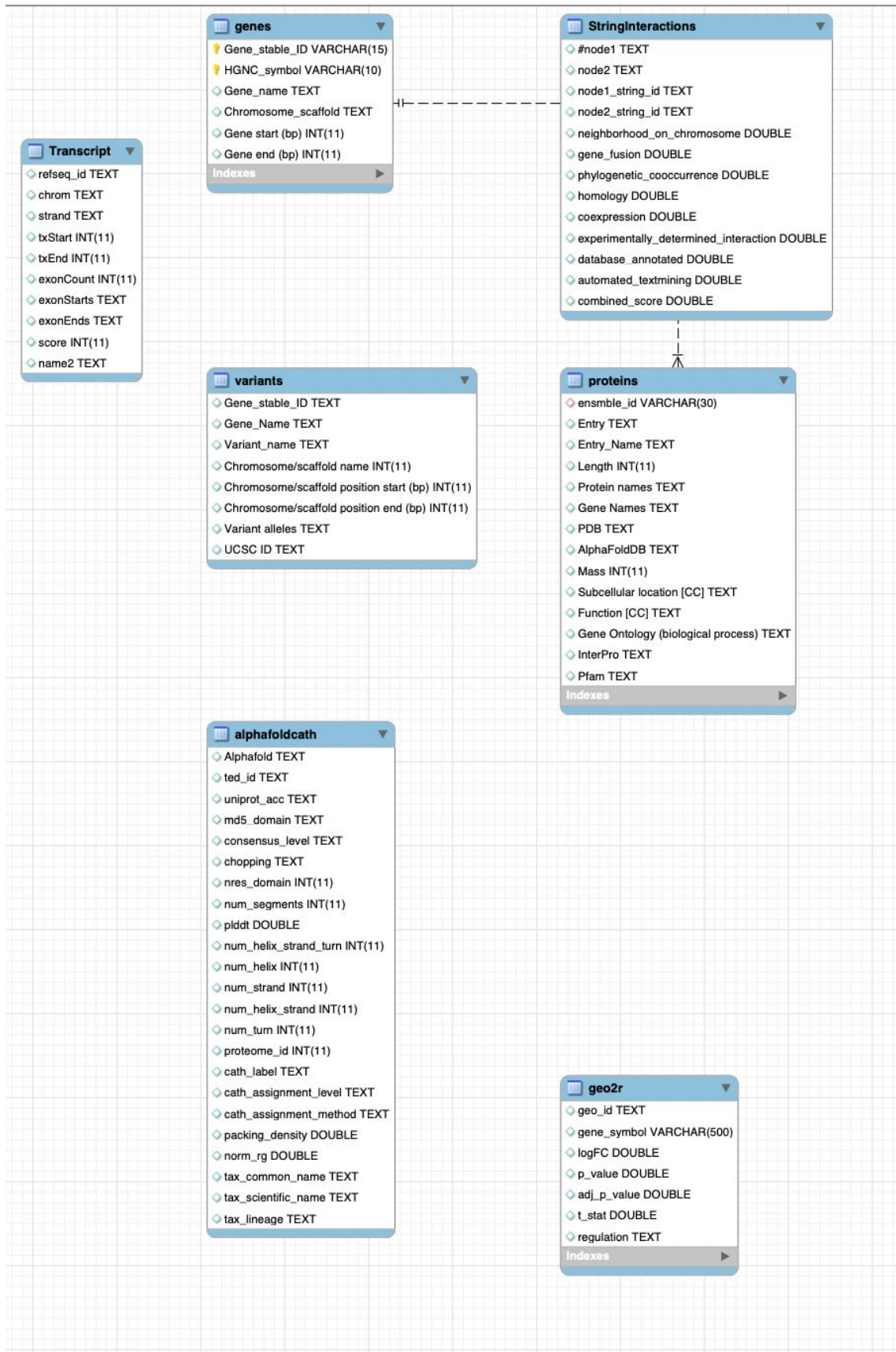


Figure 28: Final ERD exported from mysql.